



SAINT FRANCIS XAVIER COLLEGE
College of Information and Communications Technology

SMARTLAB:

A QR-BASED COMPUTER LABORATORY MONITORING SYSTEM

A Python Final Project

Presented to the
College of Information and Communications Technology
Saint Francis Xavier College

In Partial Fulfillment
of the Requirements for the Course
INTRODUCTION TO PYTHON PROGRAMMING

Presented by:
VILLACERAN, ERNAH MAE G.
MARMITO, DERIC LLOYD A.
COSTILLAS, BRET HART
CUADRA, CHARLES L.

September 2026

TABLE OF CONTENTS

Chapter 1 — Introduction.....	4
1.1 Background.....	4
1.2 Problem.....	4
1.3 Objectives.....	5
1.4 Scope and Limitations.....	5
Chapter 2 — System Design.....	7
2.1 System Features.....	7
2.2 System Flowchart.....	8
2.3 System Interface.....	9
2.4 Real-Time and Interactive Features.....	16
2.5 Computer Profile: Specifications, Status History, and Maintenance.....	17
2.6 Report Priority and Workflow.....	17
2.7 Database / Data Structure.....	18
Chapter 3 — System Development.....	22
3.1 Tools Used.....	22
3.2 System Development.....	22
3.3 Important Code.....	24
3.4 Security Hardening.....	26
Chapter 4 — Testing.....	28
Chapter 5 — Conclusion.....	31
5.1 Conclusion.....	31
5.2 Recommendations.....	31
References.....	33
Appendices.....	35
Appendix A — User Manual.....	35
Appendix B — Team Contribution.....	40

LIST OF FIGURES

Figure 2.1 — SmartLab system flow: administrator and public/student paths.....	9
Figure 2.2 — Admin dashboard with live counters, health bar, critical-reports alert, and interactive charts.....	10
Figure 2.3 — Computers page with live search, clickable filters, and inline quick-status dropdowns.....	10
Figure 2.4 — Computer profile: QR code, status control, and specifications.....	11
Figure 2.5 — Computer profile: report history, priority, and maintenance records.....	11
Figure 2.6 — Reports page with priority badges and the Open/Investigating/Under Repair/Resolved workflow.....	11
Figure 2.7 — Statistics page with interactive status and issue charts.....	12
Figure 2.8 — In-app QR Scanner page, using the device camera to jump straight to a computer's profile.....	13
Figure 2.9 — A professional, printable QR code label sheet.....	13
Figure 2.10 — The printable report log, with resolution details and signature lines for a physical record.....	14
Figure 2.11 — The branded splash/loading screen shown before login, featuring the SmartLab logo.....	14
Figure 2.12 — Public landing page shown before administrator login.....	14
Figure 2.13 — Administrator login page, now with a laboratory photo background.....	15
Figure 2.14 — The personalized welcome animation shown immediately after a successful login.	15
Figure 2.15 — The custom 404 error page.....	16

CHAPTER 1 — INTRODUCTION

1.1 Background

Computer laboratories in schools rely on manual methods to keep track of which computers are working, in use, or in need of repair (Patel & Mehta, 2026). Recent research on computer laboratory management confirms that this is not unique to any one institution: across many schools, administrators still depend on manual record-keeping and paper-based tracking, an approach that grows harder to sustain as the number of computers and users increases (Patel & Mehta, 2026). Lab attendants or instructors usually find out that a computer has a problem only after a student personally reports it, since a technician who only inspects the room periodically cannot catch every fault as it happens (Patel & Mehta, 2026), and even then, the report is often relayed verbally or written on paper, which is easy to lose or forget to act on (Patel & Mehta, 2026). This creates delays in fixing broken units, since a fault history spread across separate paper logs and spreadsheets requires slow, error-prone manual cross-referencing whenever it is needed (Patel & Mehta, 2026), and makes it difficult for lab staff to know the real-time condition of every computer in the room (Patel & Mehta, 2026).

SmartLab is proposed to address this using a web application and QR codes, a pairing already shown to give administrators fast, reliable, real-time visibility into shared equipment without manual searching (Knack Marketing, 2025). Every computer in the laboratory will be assigned a unique QR code. Scanning that code with any smartphone camera will open a mobile-friendly page — no app installation or login required — showing that computer's current status and a form to report a specific problem. Every status change and every report will be stored in a shared database, so administrators will be able to monitor and manage the condition of the entire laboratory from one dashboard, accessible from any device with a web browser. The specific gaps this proposed system is intended to close are detailed in the following section.

1.2 Problem

The current manual process of monitoring laboratory computers, described above, results in the following problems, which this study proposes to address:

- Lab staff have no quick way of knowing which computers are currently working, in use, or broken without physically checking each one.

- Problem reports made verbally or on paper are easy to lose, forget, or misplace, causing delays in repairs.
- There is no organized record of how often a specific computer breaks down, which makes it hard to identify units that may need to be replaced.
- Existing manual logs are not accessible remotely, so a technician cannot check a computer's history without being physically in the laboratory.

1.3 Objectives

General Objective

This study aims to design and develop a Flask-based web application that will use QR codes to let anyone check and report the status of individual computers in a laboratory from their own phone, and to give laboratory administrators an organized, browser-based dashboard for monitoring and resolving those reports — directly addressing the problems identified in Section 1.2.

Specific Objectives

- To generate and assign a unique, scannable QR code to every computer added to the system.
- To provide a public, mobile-friendly status page for each computer that will require no login, reachable directly by scanning its QR code.
- To allow anyone to report a specific, categorized problem on a computer, which will automatically flag that computer as "Under Repair."
- To maintain a searchable history of all problem reports and allow administrators to mark them resolved, which will automatically restore the computer's status.
- To provide a dashboard that will summarize the condition of the whole laboratory at a glance: total computers, working/under repair/offline counts, a system health percentage, and simple charts of status distribution and common issues.
- To ensure the entire system — including the public QR page — will be fully usable on desktop, tablet, and mobile phone screens.

1.4 Scope and Limitations

Scope

SmartLab will be a Flask + SQLite web application built for a single school computer laboratory, or a handful of laboratories managed together — realistically 20 to 100 computers and a similar number of active users, not an enterprise-scale deployment. It will support administrator login with hashed passwords, rate-limited against repeated attempts; adding, filtering, searching, and deleting computer records, each with optional recorded specifications (CPU, RAM, storage, OS, monitor, keyboard, mouse, internet connection); automatic QR code generation per computer, with a professional printable label sheet; a public no-login status and problem-report page per computer; a full computer status history and maintenance record log; a report management interface with priority levels and a four-stage workflow (Open, Investigating, Under Repair, Resolved), paginated for large lists; a notification indicator for recent open reports; an in-app QR scanner using the device camera; and a dashboard with summary statistics, a dedicated critical-reports indicator, and built-in charts. The system will run on a single Flask server with a local SQLite database and will be accessed through a standard web browser on any device.

Limitations

SmartLab will not send automatic external notifications such as SMS or email — new reports will only be visible inside the admin dashboard and its notification bell. It will support a single administrator account model (with a username/password-change option) rather than multiple role-based accounts, so every logged-in user will have full access to every admin feature. It is designed to run on a local network or a single deployed server rather than as a distributed, multi-server system; the rate limiter's request counters, in particular, will be kept in the memory of that one process, which is appropriate at this scale but would need a shared store such as Redis if the application were ever split across multiple server processes.

CHAPTER 2 — SYSTEM DESIGN

2.1 System Features

The interface will use a dark emerald, glassmorphism-inspired design system throughout: a deep forest-green canvas, translucent frosted-glass cards, and a soft teal glow on key status indicators — including a glowing circular gauge for overall system health and a welcome banner blending a laboratory photograph into the dashboard header. This visual language will be applied consistently across every admin page through a single shared stylesheet, so no page looks like it belongs to a different application.

SmartLab is proposed to provide the following major features:

- Landing page — explains the system and its four-step workflow before requiring any login, behind a branded splash/loading screen on first visit.
- Admin login — session-based authentication with hashed passwords, configurable session timeout, rate limiting against repeated login attempts, CSRF-protected forms, and a login/logout/failed-attempt activity log. A personalized welcome animation plays immediately after a successful login.
- Live dashboard — animated system health bar, live-updating counters (Total / Working / Under Repair / Offline / Open Reports / Critical Reports), three interactive charts (status distribution, most common issues, open vs. resolved), and a real-time activity feed — all of it refreshing automatically in the background every few seconds, with no page reload.
- Computer management — add a computer (auto-generating its QR code), instantly filter by status and search by ID, number, or lab entirely client-side, and change a computer's status directly from its card with an inline dropdown.
- Computer profile page — QR code, status changer with an optional reason, a full specifications panel, complete report history with priority, a collapsible status-history timeline, and maintenance record tracking.
- In-app QR Scanner — uses the device's own camera (phone or laptop webcam) to decode a computer's QR code from inside the admin dashboard and jump straight to its profile, with a manual ID-entry fallback.

- Public QR status page — no login required; shows a computer's live status and a categorized problem-report form; reachable only by scanning that computer's QR code or visiting its direct link.
- Report management — priority levels (Critical/High/Medium/Low) and a four-stage workflow (Open → Investigating → Under Repair → Resolved), paginated, with one-click no-reload status changes.
- Notification bell — polls for the current open-report count and animates when it increases, so it stays accurate even while the administrator is on a different page.
- Statistics page — the same interactive charts as the dashboard, focused on a dedicated statistics view.
- QR label printing — a professional, branded, printable label sheet for every computer or a single one, with dashed cut-lines.
- Printable report log — a formal, signature-ready record of every problem report, showing what was reported, how it was resolved, who fixed it, and when, filterable by computer or by status, for keeping a physical proof of record.
- Settings — ten sections covering General, Account (username/password), Laboratory reference info, Appearance, QR code generation, Security, System maintenance, Notifications, About, and a confirmation-gated Danger Zone.
- Custom error pages — branded 404, 403, 429, and 500 pages that never expose technical details.
- Responsive design — the sidebar collapses into a mobile drawer, computer cards and charts stack vertically, and tables reflow into stacked rows on small screens.

2.2 System Flowchart

SmartLab will have two independent entry paths: the administrator path (login required) and the public path (QR scan, no login). The diagram below, drawn using standard flowchart notation with the accompanying legend, illustrates both paths at a high level; in practice, a filed report will move through the more granular Open → Investigating → Under Repair → Resolved workflow described in Section 2.6, rather than jumping directly from a new report to a resolved one.

Symbol	Name	Function
	Start/end	An oval represents a start or end point
	Arrows	A line is a connector that shows relationships between the representative shapes
	Input/Output	A parallelogram represents input or output
	Process	A rectangle represents a process
	Decision	A diamond indicates a decision

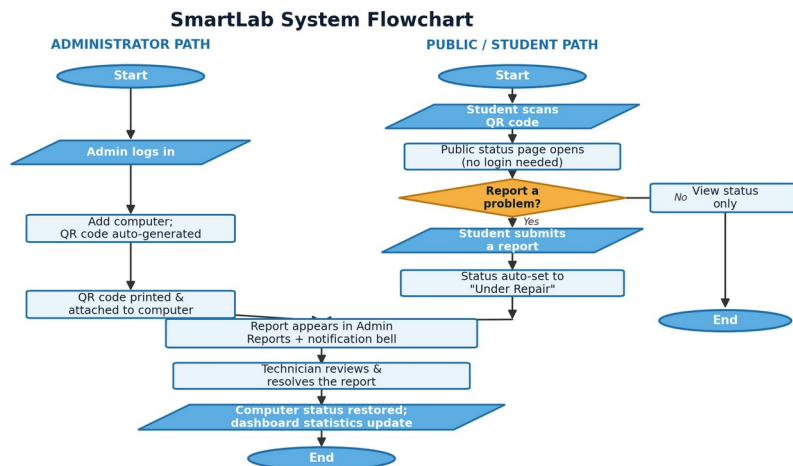


Figure 2.1 — SmartLab system flow: administrator and public/student paths.

2.3 System Interface

The following are mockups illustrating the proposed system interface.

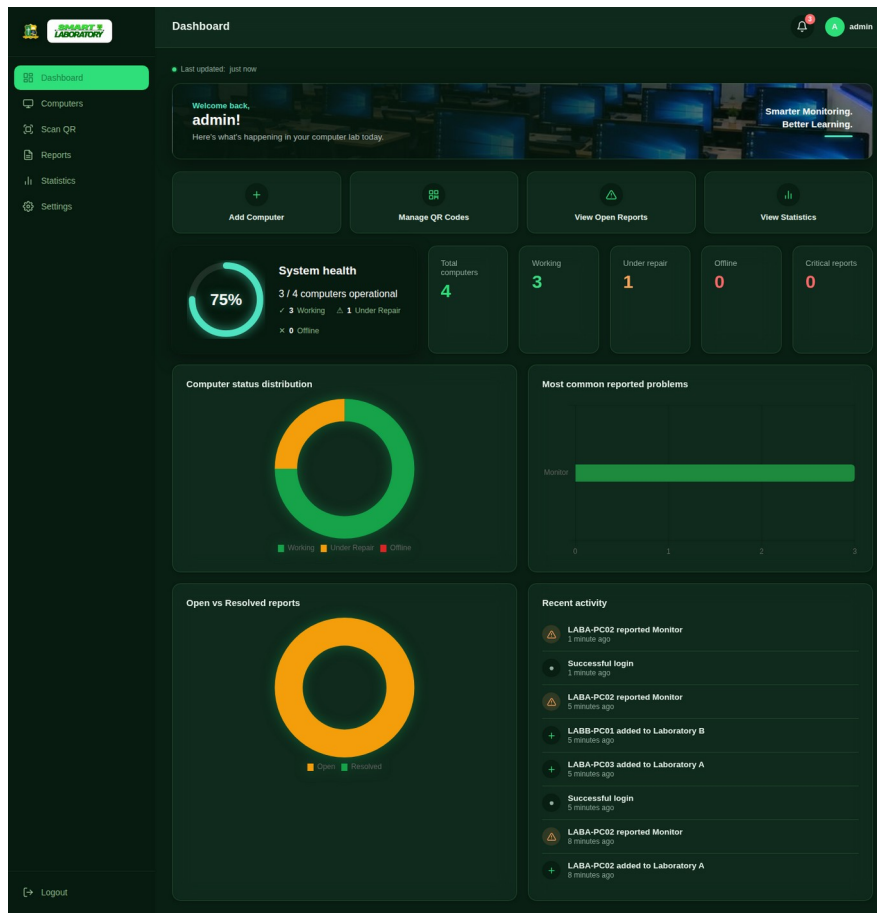


Figure 2.2 — Admin dashboard with live counters, health bar, critical-reports alert, and interactive charts.

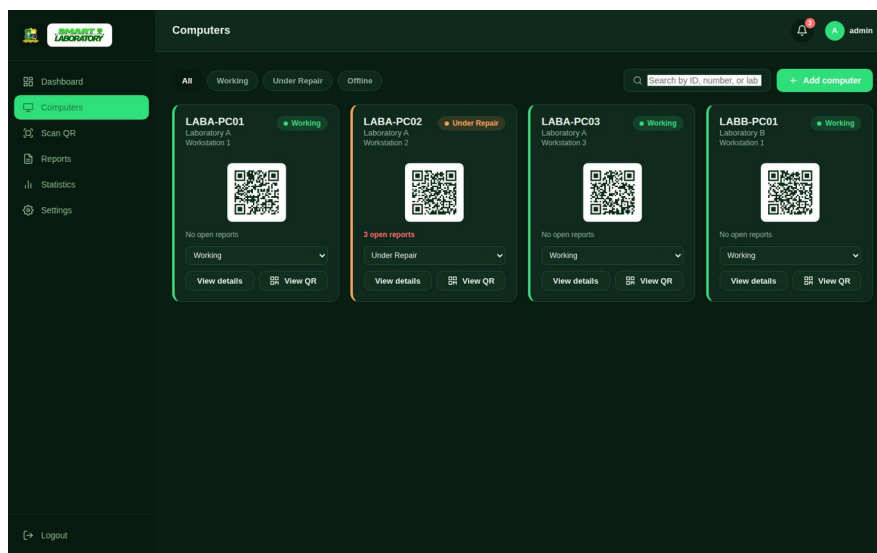


Figure 2.3 — Computers page with live search, clickable filters, and inline quick-status dropdowns.

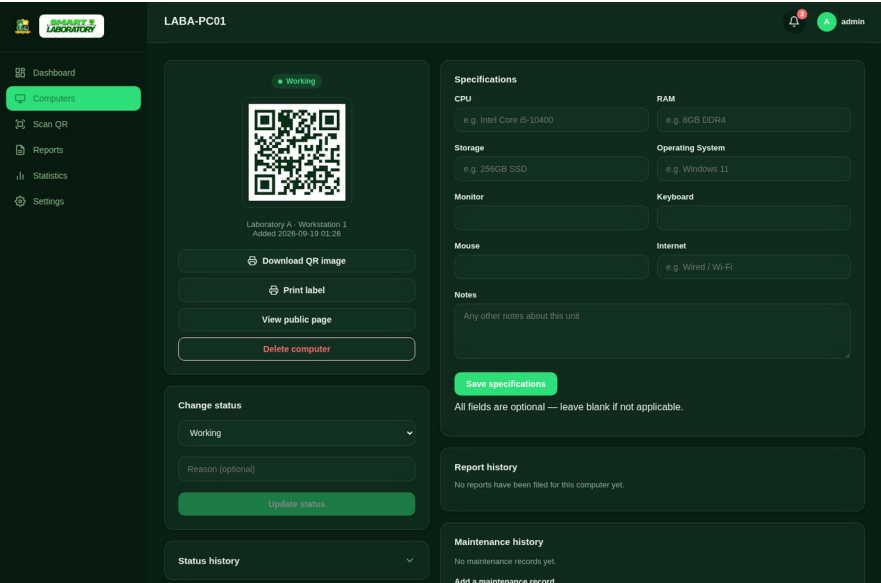


Figure 2.4 — Computer profile: QR code, status control, and specifications.

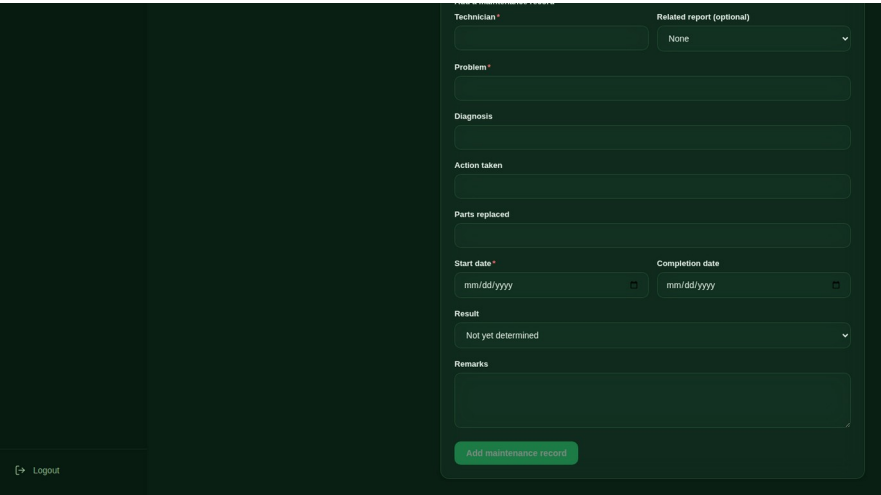


Figure 2.5 — Computer profile: report history, priority, and maintenance records.

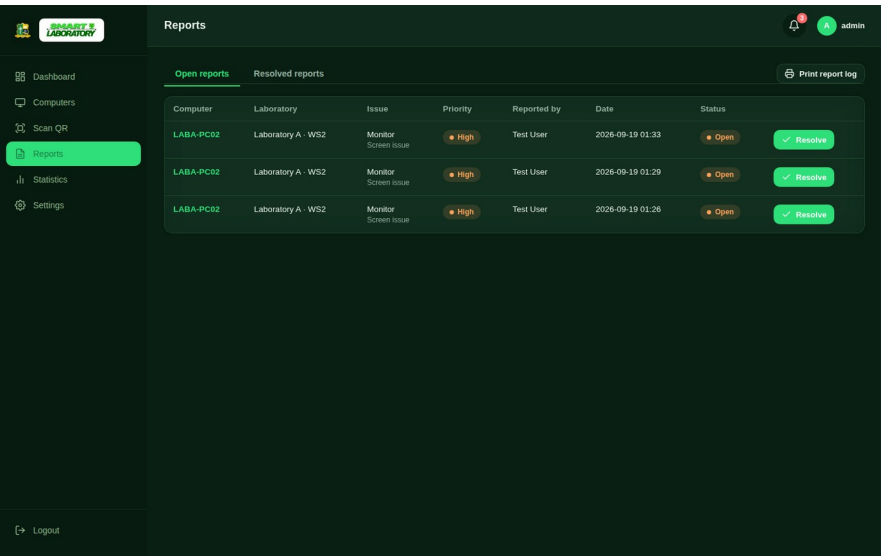


Figure 2.6 — Reports page with priority badges and the Open/Investigating/Under Repair/Resolved workflow.

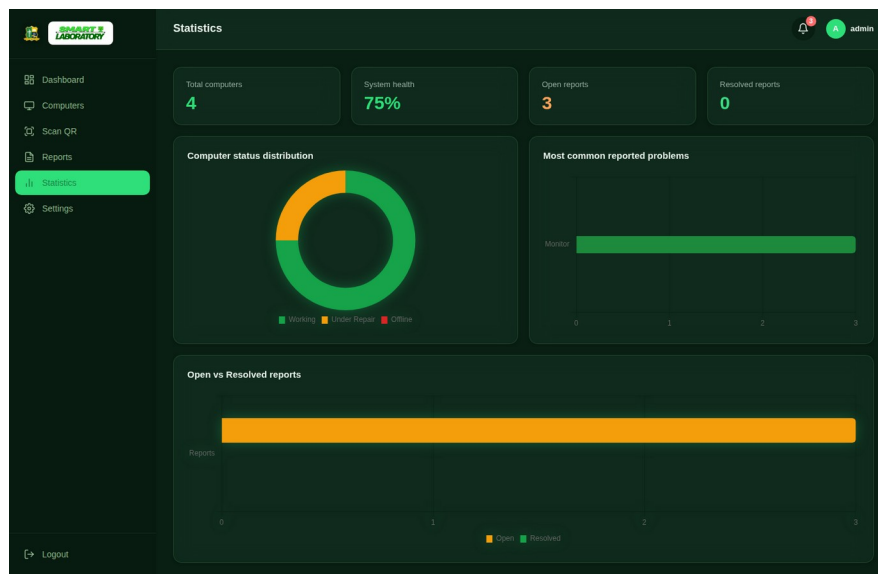


Figure 2.7 — Statistics page with interactive status and issue charts.

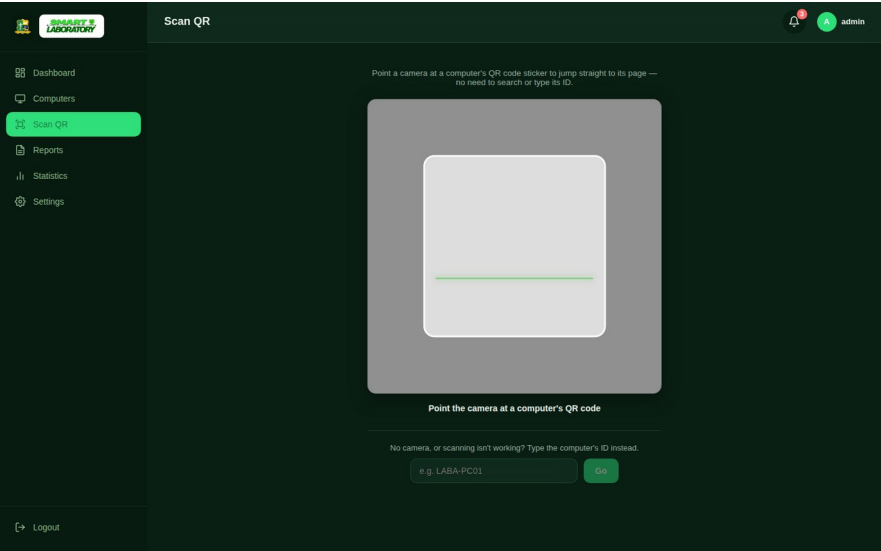


Figure 2.8 — In-app QR Scanner page, using the device camera to jump straight to a computer's profile.

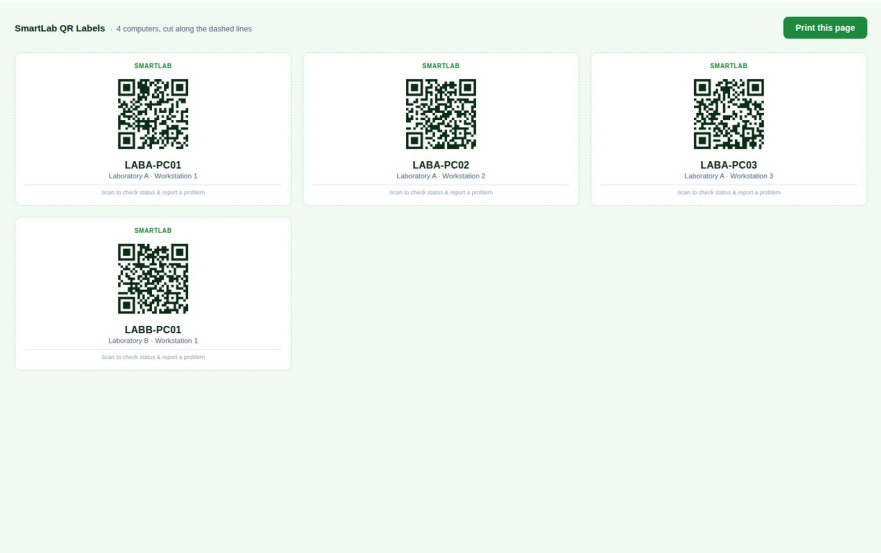


Figure 2.9 — A professional, printable QR code label sheet.

SmartLab Report Log - 3 reports Print this page

SMARTLAB
Computer Problem Report Log

Printed: 2026-09-19 01:34 AM
Filter: All reports

COMPUTER	DATE REPORTED	REPORTED ISSUE	RESOLUTION / ACTION TAKEN	FIXED BY	DATE RESOLVED	STATUS
LABA-PC02 Laboratory A - WS2	2026-09-19 01:33	Monitor Screen issue	Not yet recorded	---	---	Open
LABA-PC02 Laboratory A - WS2	2026-09-19 01:29	Monitor Screen issue	Not yet recorded	---	---	Open
LABA-PC02 Laboratory A - WS2	2026-09-19 01:26	Monitor Screen issue	Not yet recorded	---	---	Open

Prepared by (print name & signature) _____
Reviewed by (print name & signature) _____
Date _____

Figure 2.10 — The printable report log, with resolution details and signature lines for a physical record.

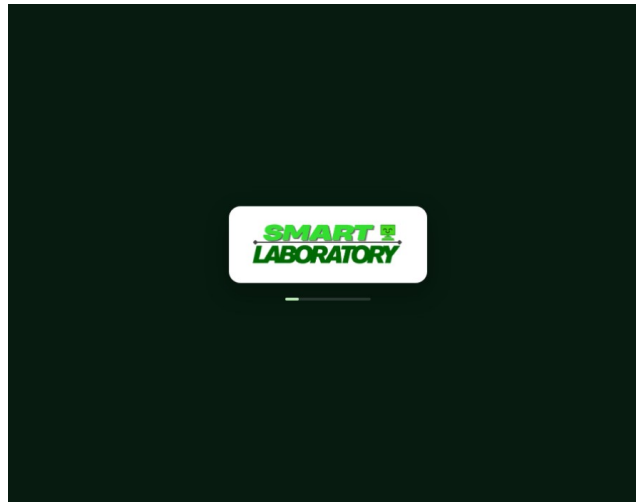


Figure 2.11 — The branded splash/loading screen shown before login, featuring the SmartLab logo.

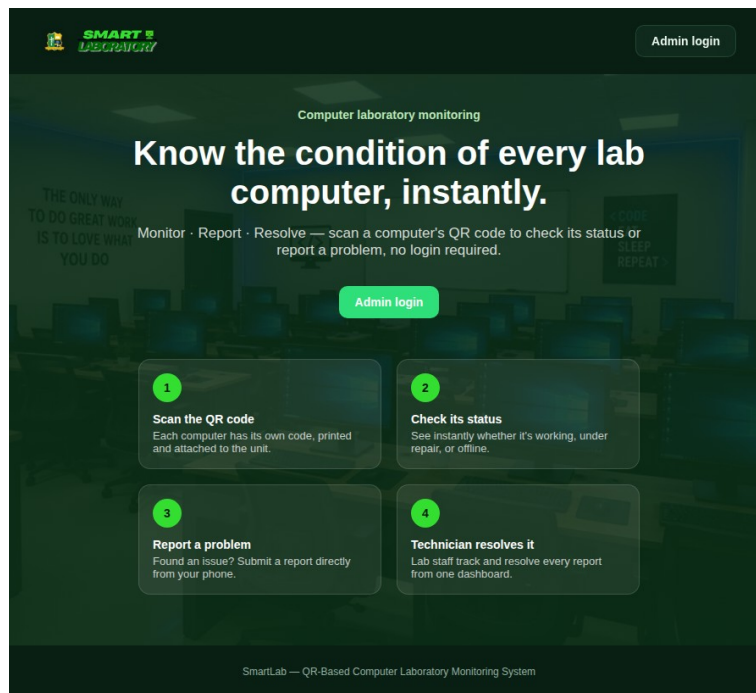


Figure 2.12 — Public landing page shown before administrator login.

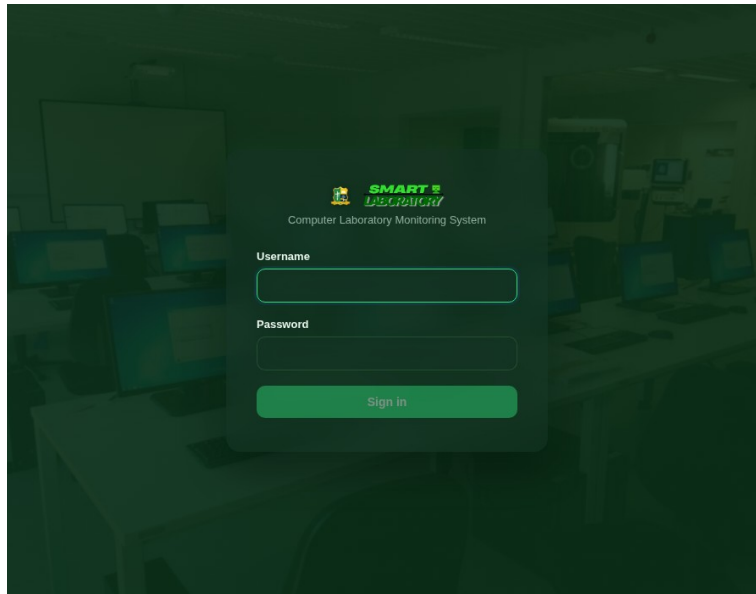


Figure 2.13 — Administrator login page, now with a laboratory photo background.

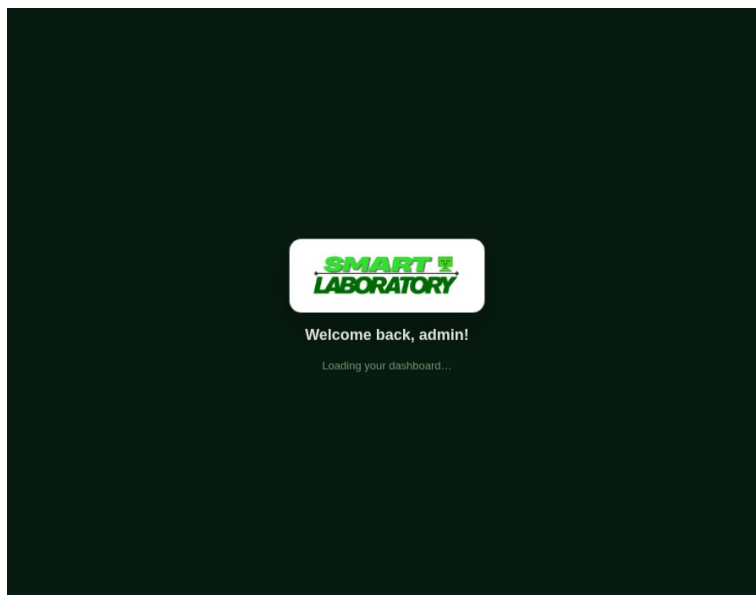


Figure 2.14 — The personalized welcome animation shown immediately after a successful login.

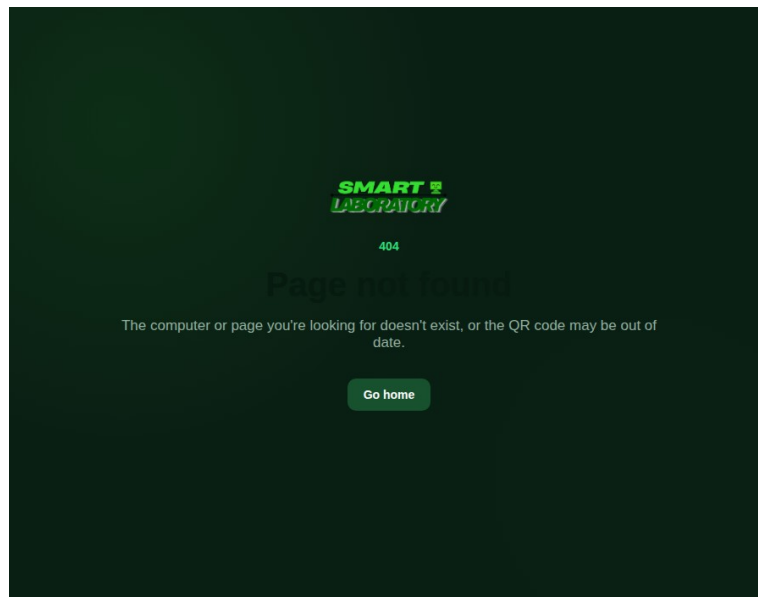


Figure 2.15 — The custom 404 error page.

2.4 Real-Time and Interactive Features

A core requirement for the proposed version of SmartLab is that the interface will respond visually whenever the underlying data changes, instead of requiring a manual page refresh. This will be implemented as follows:

- A single combined endpoint, `/api/dashboard-data`, returns the current statistics (including the critical-reports count), issue counts, and recent activity as JSON, computed directly from the database on every request.
- A small JavaScript file (`live.js`) polls that endpoint every seven seconds from any admin page, updates the notification bell, and dispatches a custom `smartlab:dashboard-data` browser event.
- The Dashboard and Statistics pages listen for that event and update their charts, animated counters, and health bar in place — numbers count up or down smoothly rather than jumping instantly, and the critical-reports card visibly pulses red whenever that count is above zero.
- Actions the administrator takes directly — changing a computer's status from its card, or resolving a report — call small JSON API endpoints with the Fetch API, so the on-screen result appears immediately rather than waiting for the next poll.
- Every one of these values is read from the database at the moment of the request; nothing shown on the dashboard is hard-coded or simulated.

2.5 Computer Profile: Specifications, Status History, and Maintenance

Each computer's profile page is designed to go well beyond a simple status display, to give administrators a genuine maintenance record for every machine.

Specifications

Administrators will be able to optionally record a computer's CPU, RAM, storage, operating system, monitor, keyboard, mouse, and internet connection, plus free-text notes. Every field will be optional, so this feature will not require existing computer records to be filled in before they can still be viewed and used normally.

Status history

Every status change — whether made manually by an administrator (optionally with a typed reason) or automatically because a report was filed or resolved — will be permanently recorded with the old status, the new status, who or what made the change, and a timestamp. This history will never be overwritten; it will be shown as a collapsible timeline on the computer's profile.

Maintenance records

Administrators will be able to log a full maintenance record against a computer: the technician, the problem, the diagnosis, the action taken, any parts replaced, start and completion dates, the result, and remarks. Each record can optionally be linked to the specific report that prompted it. The most recent maintenance date will be shown at the top of the computer's maintenance history.

2.6 Report Priority and Workflow

Reports will carry a priority level and move through a more realistic multi-stage workflow, rather than only ever being "Open" or "Resolved."

Priority

Every report will be assigned a priority — Critical, High, Medium, or Low — shown as a color-coded badge (red, orange, yellow, and green respectively) alongside its text label, so meaning will never be conveyed by color alone. New reports will default to Medium priority and can be re-prioritized by an administrator at any time; the Reports page will sort open reports with the most urgent priority first.

Workflow

A report will move through Open → Investigating → Under Repair → Resolved. Any of the first three states will count as "open" for dashboard and notification purposes. Reaching Resolved will automatically restore the computer's status to Working, but only if that computer has no other reports still open — the same safeguard a simpler binary Open/Resolved workflow would need, generalized here to the full four-stage system.

2.7 Database / Data Structure

SmartLab will store its data in a local SQLite database file (instance/smartlab.db). The core tables are proposed as follows:

computers

Field	Type	Description
id	INTEGER (Primary Key)	Auto-generated internal row ID
computer_id	TEXT (Unique)	The public-facing ID encoded in the QR code, e.g. LABA-PC01
lab	TEXT	Which laboratory the computer belongs to, e.g. Laboratory A
number	INTEGER	Workstation number within that laboratory
status	TEXT	Working / Under Repair / Offline
qr_path	TEXT	Relative path to the generated QR code image under static/
date_added	TEXT	Date and time the computer was registered
cpu, ram, storage, os,	TEXT (all optional)	Recorded specifications — every field may be left blank
monitor, keyboard, mouse,		without affecting any other part of the computer's record
internet, specs_notes		

reports

Field	Type	Description
id	INTEGER (Primary Key)	Auto-generated report number
computer_id	TEXT (Foreign Key)	Which computer the report is about
issue_type	TEXT	One of eight predefined issue categories
description	TEXT	Optional free-text description of the problem
reporter	TEXT	Name of the person reporting (optional; defaults to "Anonymous")
date_reported	TEXT	Date and time the report was filed
status	TEXT	Open / Investigating / Under Repair / Resolved
date_resolved	TEXT	Date and time the report was marked resolved (if applicable)
priority	TEXT	Critical / High / Medium / Low — defaults to Medium

users

Field	Type	Description
id	INTEGER (Primary Key)	Auto-generated user number
username	TEXT (Unique)	Administrator login username
password_hash	TEXT	Salted hash of the password, generated with Werkzeug — the plain password is never stored

computer_status_history

Field	Type	Description
id	INTEGER (Primary Key)	Auto-generated entry number
computer_id	TEXT (Foreign Key)	Which computer this entry belongs to
old_status	TEXT	The status before this change (blank for a computer's very first entry)
new_status	TEXT	The status after this change
reason	TEXT	Optional free-text reason, typed by an administrator or auto-filled (e.g. "New report: Monitor")
changed_by	TEXT	Username, or "System"/"Public report" when the change happened automatically
timestamp	TEXT	Date and time of the change

A new row is added every time a computer's status actually changes; if a status is "changed" to the value it already had, no row is written, to keep the history meaningful rather than noisy.

maintenance

Field	Type	Description
id	INTEGER (Primary Key)	Auto-generated record number
computer_id	TEXT (Foreign Key)	Which computer this record is about
report_id	INTEGER (Foreign Key, optional)	The report that prompted this maintenance, if any
technician	TEXT	Who performed the work
problem, diagnosis,	TEXT	Free-text description of the issue, its cause, what was done,
action_taken, parts_replaced		and any parts replaced
start_date, completion_date	TEXT	When the work began and finished (completion optional)
result, remarks	TEXT	Outcome (e.g. Resolved) and any additional notes

Three further tables support the Settings page without altering any of the tables above:

settings

Field	Type	Description
setting_key	TEXT (Primary Key)	Name of the setting, e.g. "theme" or "qr_size"
setting_value	TEXT	Its current value, stored as plain text

laboratories

Field	Type	Description
id	INTEGER (Primary Key)	Auto-generated row ID
name	TEXT (Unique)	Laboratory name, e.g. "Laboratory A"
location	TEXT	Optional physical location, e.g. "Main Building — Room 204"
description	TEXT	Optional free-text description

This table is reference information only — the computer count shown for each lab is computed live from the computers table, and deleting a laboratory here never deletes or changes any computer.

activity_log

Field	Type	Description
id	INTEGER (Primary Key)	Auto-generated entry number
event_type	TEXT	Human-readable description, e.g. "PC-012 status changed to Offline"
username	TEXT	Who performed the action, where applicable
timestamp	TEXT	Date and time of the event
category	TEXT	Drives which icon the activity feed shows
computer_id	TEXT	Which computer the event relates to, where applicable

This table powers both the dashboard's Recent Activity feed and the Security tab's login history. It never stores passwords.

Indexes

To keep filtering and searching fast as the number of computers and reports grows, indexes were added on the columns actually used in WHERE and ORDER BY clauses: computers.status, computers.lab, reports.computer_id, reports.status, reports.priority, and reports.date_reported, plus the computer_id column on both the status-history and maintenance tables. Deliberately not every column is indexed — over-indexing slows down writes for little benefit on a database this size.

CHAPTER 3 — SYSTEM DEVELOPMENT

3.1 Tools Used

- Python 3 — main programming language
- Flask — micro web framework used to build the routes and server-side logic, including a small JSON API
- Jinja2 — Flask's built-in templating engine, used to build reusable HTML templates
- SQLite (via Python's built-in sqlite3 module) — stores all computer records, reports, admin accounts, settings, activity history, status history, and maintenance records, with indexes on the columns actually used for filtering
- Werkzeug — provides password hashing (`generate_password_hash` / `check_password_hash`) for secure admin login
- Flask-WTF — provides CSRF protection for every form and JSON API call that changes data
- Flask-Limiter — rate-limits the login form and the public report form against repeated automated requests
- qrcode and Pillow (PIL) — generate the QR code image for every computer
- jsQR — decodes a QR code from the device camera entirely in the browser, for the in-app QR Scanner page; like Chart.js, it is bundled with the project rather than loaded from a CDN
- Chart.js — renders the interactive status-distribution, issue-frequency, and open/resolved charts on the Dashboard and Statistics pages; bundled with the project itself rather than loaded from an external CDN, so the charts still work without an internet connection during a live demonstration
- HTML5, CSS3, and vanilla JavaScript (Fetch API) — the public-facing interface, the responsive layout and mobile navigation drawer, and the live-update/polling logic
- Google Fonts (Inter) — typography used throughout the interface
- Visual Studio Code — code editor used throughout development

3.2 System Development

Development will start with the database design, since every route will depend on how computers, reports, and users are stored. The core tables (computers, reports, users) will be created first in `database.py`, along with helper functions for every operation the application needs — adding a computer, updating a status, filing a report, resolving a report, and computing dashboard statistics — so that no SQL will be written directly inside the Flask routes. Further tables (settings, laboratories, `activity_log`, `computer_status_history`, and maintenance) will be added additively over time, each as a genuine schema migration that checks for the table or column's existence before creating it, so that upgrading an existing installation will never lose data already stored.

Next, the Flask routes will be built in `app.py`: first the admin-only routes protected by a `login_required` decorator, then the public routes that will intentionally require no authentication (the landing page, the login page, and the per-computer QR status page). The QR code itself will be designed to encode a full URL to that computer's public page, so that scanning it with any phone camera will open the page directly with no extra app needed.

The interface will then be built as a set of Jinja2 templates sharing a common base layout for admin pages (a sidebar and top bar) and a separate, simpler layout for public pages. A single CSS file will define the entire visual design as a set of reusable tokens (colors, spacing, shadows) so that every page — admin or public — will stay visually consistent.

The application will then be made live and interactive: a JSON API will be added alongside the existing page routes, and a shared JavaScript file (`live.js`) will poll that API every few seconds and animate any values that changed. `Chart.js` will be integrated for the status, issue, and open/resolved visualizations, and small custom modal dialogs and toast notifications will replace full-page reloads for actions like changing a computer's status and resolving a report.

A later phase will substantially expand what a computer record and a report can represent: optional specifications, a permanent status-history log, maintenance record tracking, report priority levels, and a four-stage report workflow. An in-app QR Scanner, a redesigned printable QR label sheet, and a branded splash screen and login-welcome animation will follow. The final phase will be a security and reliability pass: CSRF protection across every form and API call, rate limiting on the two endpoints that accept input from anyone (not just logged-in administrators), and custom branded error pages so that a real error will never expose

a raw stack trace. Responsiveness will be verified throughout, across desktop and mobile widths.

Each feature will be tested as it is built by actually running the Flask development server and exercising it end to end, rather than only reading the code.

3.3 Important Code

Only the key parts of the proposed program are illustrated here to show how the design will be implemented; the complete source code will be provided separately as the project's source files upon completion.

Generating a QR code that links to a computer's public page (qr_utils.py)

```
def generate_qr(computer_id, base_url, box_size=8):
    target_url = f"{base_url.rstrip('/')}/qr/{computer_id}"
    qr = qrcode.QRCode(border=2, box_size=box_size)
    qr.add_data(target_url)
    qr.make(fit=True)
    img = qr.make_image(fill_color="#0F26A6", back_color="white")
    ...
    img.save(path)
    return f"qrcores/{filename}"
```

Recording every status change without overwriting history (database.py)

```
def update_status(computer_id, new_status, reason=None, changed_by=None):
    conn = get_connection()
    old_row = conn.execute(
        "SELECT status FROM computers WHERE computer_id=?", (computer_id,)
    ).fetchone()
    old_status = old_row["status"] if old_row else None
    conn.execute("UPDATE computers SET status=? WHERE computer_id=?",
                  (new_status, computer_id))
    if old_status != new_status: # only log a real change, not a no-op
        conn.execute(
            "INSERT INTO computer_status_history "
            "(computer_id, old_status, new_status, reason, changed_by, "
timestamp) "
            "VALUES (?, ?, ?, ?, ?, ?)",
            (computer_id, old_status, new_status, reason,
changed_by, ...),
        )
        conn.commit()
        conn.close()
```

Generalizing report resolution into a four-stage workflow (database.py)

```
def update_report_status(report_id, new_status, changed_by=None):
    ...
    if new_status == "Resolved":
        conn.execute("UPDATE reports SET status=?, date_resolved=? WHERE
id=?", ...)
```

```

    else:
        conn.execute("UPDATE reports SET status=? WHERE id=?",
(new_status, report_id))
        ...
    if new_status == "Resolved":
        remaining = get_open_report_count(computer_id)
        if remaining == 0:
            update_status(computer_id, "Working", changed_by=changed_by or
"System")
def resolve_report(report_id):
    """Kept for backward compatibility - jumps a report straight to
Resolved."""
    return update_report_status(report_id, "Resolved")

```

The combined live-dashboard API endpoint (app.py)

Every value below will be read from the database at request time, so that nothing shown on the dashboard will be invented or hard-coded.

```
@app.route("/api/dashboard-data")
@login_required
def api_dashboard_data():
    stats = db.get_dashboard_stats() # includes critical_reports
    issue_counts = db.get_issue_type_counts()
    activity = db.get_recent_activity(limit=8)
    return jsonify({"stats": stats, "issue_counts": [...],
                    "activity": [...], "server_time": ...})
```

3.4 Security Hardening

A dedicated pass will be made over the whole application specifically to harden it, rather than treating security as a byproduct of building features.

CSRF protection (Flask-WTF)

Every form and JSON API call that changes data will be protected against cross-site request forgery. Rather than hand-editing dozens of individual form tags, a small shared script will read a token from a meta tag already present on every page and inject it into every POST form automatically on page load, and the same token will be attached as a header to the handful of raw fetch() calls that will not go through a form (the quick-status dropdown and the AJAX resolve button).

```
app.config["WTF_CSRF_TIME_LIMIT"] = None
csrf = CSRFProtect(app)

// forms.js - runs on every page:
function injectCsrfTokens() {
    var token = getCsrfToken();
    document.querySelectorAll("form").forEach(function (form) {
        if (form.method !== "post" || form.querySelector('[name=csrf_token]'))
            return;
        var input = document.createElement("input");
        input.type = "hidden"; input.name = "csrf_token"; input.value = token;
        form.appendChild(input);
    });
}
```

Rate limiting (Flask-Limiter)

Rate limiting will be applied narrowly rather than globally: only the login form (against repeated password guessing) and the public report form (against spam) will accept input from

people who are not already authenticated, so those will be the two endpoints actually worth limiting.

```
limiter = Limiter(get_remote_address, app=app, storage_uri="memory://")

@app.route("/login", methods=["GET", "POST"])
@limiter.limit("10 per minute", methods=["POST"])
def login(): ...

@app.route("/qr/<computer_id>/report", methods=["POST"])
@limiter.limit("20 per hour")
def submit_report(computer_id): ...
```

Custom error pages

404, 403, 429, and 500 will all render a branded SmartLab page rather than Flask's default error output, and none of them will expose a stack trace, a file path, or any other technical detail to the visitor.

```
@app.errorhandler(429)
def too_many_requests(e):
    return render_template("429.html"), 429
```

CHAPTER 4 — TESTING

Each feature was tested by actually running the Flask application and using it end to end, rather than only reading the code. This included adding computers under two laboratories, recording specifications and maintenance history, decoding a generated QR code to confirm it linked to the correct public page, submitting reports as an anonymous visitor and moving them through the full priority and workflow system, and confirming that the dashboard, statistics, and notification bell all updated immediately after each action. The live-update mechanism specifically was tested by opening the dashboard, changing a computer's status through a separate request (simulating a second administrator or device), and confirming the counters, health bar, chart, and activity feed all updated on their own within one polling cycle — without the page ever being reloaded. The security additions — CSRF protection and rate limiting — and the in-app QR Scanner were each tested by deliberately trying to break or bypass them, not just by confirming the happy path worked.

Problems Encountered

- Problem: Adding a computer logged a "computer added" activity entry twice, once from inside the database layer and once from a redundant call added later in the Flask route. Solution: The redundant call was removed, keeping activity logging in exactly one place per action so every event appears in the feed exactly once.
- Problem: A screenshot tool used during development could not run the Chart.js charts at all — the canvases came out completely blank — which initially looked like a serious bug in the live dashboard. Solution: The same page was opened in an actual Chromium browser (via Playwright) and the charts rendered correctly with real, live data; the issue was traced to that particular tool's outdated JavaScript engine rather than the application, since Chart.js only requires the modern browser features already present in Chrome, Firefox, Safari, and Edge. This was a useful reminder to verify visual bugs in a real, current browser before concluding the application itself is broken.
- Problem: A CSS layout using `position: fixed` for the sidebar rendered as completely blank when captured with an older WebKit-based rendering tool used earlier in testing. Solution: The sidebar was rebuilt using standard flexbox layout (a normal flex child stretching to full height) for the desktop view, which is both more broadly compatible

and simpler, while keeping position: fixed only for the mobile off-canvas drawer, where it is actually needed.

- Problem: On mobile screens, hiding the sidebar with a CSS transform still left its layout space reserved, squeezing all page content into a narrow column. Solution: The sidebar is now taken out of normal document flow with position: fixed specifically inside the mobile media query, so it overlays the page instead of pushing it, matching the standard mobile "drawer" navigation pattern.
- Problem: Typing a computer ID in a different letter case (e.g. "laba-pc01" versus "LABA-PC01") could have created duplicate-looking records. Solution: Computer IDs are automatically converted to uppercase before being saved or looked up, in both the add-computer form and the public report form.
- Problem: Filing a new report while a computer already had other open reports could incorrectly mark it "Working" again as soon as any one report was resolved. Solution: Resolving a report now checks whether any other open reports remain for that computer before restoring its status, using `get_open_report_count()`.
- Problem: The Flask secret key, used to secure login sessions, was at risk of being hard-coded. Solution: The application reads it from the `SMARTLAB_SECRET_KEY` environment variable, only falling back to a clearly-labeled development key if that variable is not set.
- Problem: Every destructive action in Settings (deleting reports, resetting computers) needed a safeguard stronger than a browser pop-up, which can be dismissed accidentally. Solution: Each Danger Zone action requires the administrator to type the word "DELETE" into a text field, checked on the server itself, before anything is removed.
- Problem: When report priority and a four-stage workflow were introduced, the badge that displays a report's status still only recognized "Open" and treated every other value — including the new "Investigating" and "Under Repair" states — as "Resolved." The same narrow check also hid the Resolve button for any report not in exactly the "Open" state. Solution: Both were widened to recognize all four states correctly (`status != "Resolved"` for "is this report still active"), and this exact pattern was specifically re-checked everywhere else status was compared, since a narrow equality check like this is an easy mistake to repeat.

- Problem: A new schema migration must never run against a database that already has real data and quietly damage it. Solution: Before trusting the migration, a copy of the old three-table schema was built by hand and seeded with real rows, the new `initialize_database()` was run against it, and every original row was confirmed byte-for-byte unchanged afterward, with the new optional columns simply blank.
- Problem: After adding CSRF protection, it was not obvious whether real users would actually still be able to submit forms, or whether the protection was strong enough to matter. Solution: Both directions were tested directly — a forged POST request with no token was confirmed to be rejected (HTTP 400), and a real browser session was confirmed to submit successfully, because the token is injected into the form automatically before submission.
- Problem: Confirming rate limiting actually worked required deliberately trying to break it. Solution: Twelve rapid login attempts were fired in a row; exactly ten succeeded (matching the configured "10 per minute" limit) and the remaining two received the real custom 429 page, not Flask's default error output.
- Problem: The in-app QR Scanner could not be tested with a real camera in this environment. Solution: Chromium's fake-video-capture feature was used to feed the browser an actual video of a real generated QR code, and the scanner was confirmed to decode it and navigate to the correct computer's page in under a second — as close to a genuine hardware test as is possible without a physical device.

CHAPTER 5 — CONCLUSION

5.1 Conclusion

The SmartLab web application successfully met its objective of giving a computer laboratory a fast, organized way to monitor and report on the condition of individual computers using QR codes. Because it is browser-based, any student can check a computer's status or file a report using only their own phone's camera and browser, with no app installation and no login. Administrators, in turn, get a live dashboard that reflects the true condition of the laboratory at all times, a genuine maintenance record for every machine, and a report workflow that mirrors how a real repair actually progresses — rather than a static page that only shows what was true at the last manual refresh.

Building this project also demonstrated how a web framework (Flask), a templating engine (Jinja2), a database (SQLite), a small JSON API, and a small set of supporting libraries (Werkzeug and Flask-WTF for security, Flask-Limiter for abuse prevention, qrcode and jsQR for QR generation and scanning, Chart.js for visualization) can be combined into a single, cohesive, defense-ready system. It also reinforced the importance of testing each route and each user path independently before relying on them together, of verifying visual behavior in a real, current browser rather than trusting a single testing tool, and of actually trying to break a new security control rather than assuming it works because the code looks correct.

5.2 Recommendations

- Deploy the application behind a production WSGI server such as Gunicorn, with a reverse proxy such as Nginx, instead of Flask's built-in development server.
- Add role-based accounts (e.g. separate technician and administrator roles) instead of a single shared admin login.
- Add automatic email or SMS notifications to technicians whenever a new report is filed, in addition to the in-app notification bell.
- Replace the fixed-interval polling used for live updates with WebSockets if the laboratory grows large enough that polling every few seconds becomes noticeably heavier on the server.

- Move the rate limiter's request counters from in-process memory to a shared store such as Redis if the application is ever deployed across multiple server processes, since each process currently keeps its own separate counters.
- Extend search and pagination to the Computers page for very large laboratories; it currently loads every computer at once for instant client-side filtering, which is fast at the 20–100 computer scale this version targets but would benefit from the same page-based approach used on the Reports page at a much larger scale.

REFERENCES

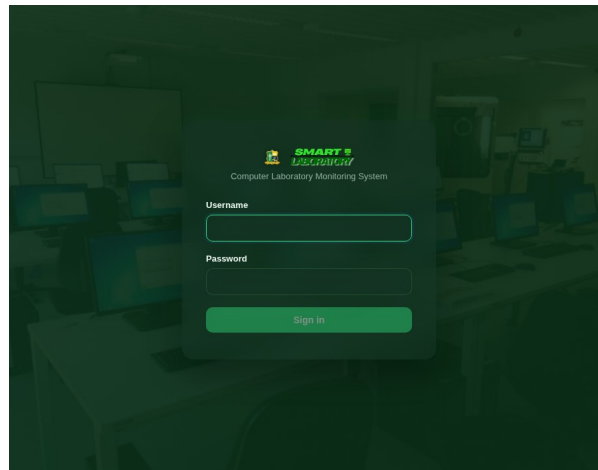
- Chart.js Contributors. (n.d.). Chart.js documentation (Version 4). Retrieved September 2026, from <https://www.chartjs.org/docs/>
- Wolfe, C. (n.d.). jsQR: A pure JavaScript QR code reading library [Computer software]. GitHub. Retrieved September 2026, from <https://github.com/cozmo/jsQR>
- Google Fonts. (n.d.). Inter [Font]. Google. Retrieved September 2026, from <https://fonts.google.com/specimen/Inter>
- Knack Marketing. (2025). QR code asset tracking guide: Benefits, features & no-code setup. Knack. <https://www.knack.com/blog/qr-code-asset-tracking-guide/>
- Pallets Projects. (n.d.). Flask-WTF documentation. Retrieved September 2026, from <https://flask-wtf.readthedocs.io/>
- Mozilla Developer Network. (n.d.). CSS: Cascading style sheets. MDN Web Docs. Retrieved September 2026, from <https://developer.mozilla.org/en-US/docs/Web/CSS>
- Mozilla Developer Network. (n.d.). Fetch API. MDN Web Docs. Retrieved September 2026, from https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- Mozilla Developer Network. (n.d.). MediaDevices: getUserMedia() method. MDN Web Docs. Retrieved September 2026, from <https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia>
- Mozilla Developer Network. (n.d.). Flexible box layout. MDN Web Docs. Retrieved September 2026, from https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_flexible_box_layout
- Mozilla Developer Network. (n.d.). CSS grid layout. MDN Web Docs. Retrieved September 2026, from https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout
- OWASP Foundation. (n.d.). Cross-Site Request Forgery prevention cheat sheet. OWASP Cheat Sheet Series. Retrieved September 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

- OWASP Foundation. (n.d.). Password storage cheat sheet. OWASP Cheat Sheet Series. Retrieved September 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- Patel, S. J., & Mehta, H. (2026). Computer lab management system: A comprehensive review. *International Journal of Research Publication and Reviews*, 7(4), 1495–1500. <https://doi.org/10.55248/gengpi.07.0426.10811>
- Pallets Projects. (n.d.). Flask documentation. Retrieved September 2026, from <https://flask.palletsprojects.com/>
- Pallets Projects. (n.d.). Jinja documentation. Retrieved September 2026, from <https://jinja.palletsprojects.com/>
- Pallets Projects. (n.d.). Werkzeug documentation. Retrieved September 2026, from <https://werkzeug.palletsprojects.com/>
- Python Software Foundation. (n.d.). Python 3 documentation. Retrieved September 2026, from <https://docs.python.org/3/>
- Python Software Foundation. (n.d.). sqlite3 — DB-API 2.0 interface for SQLite databases. Python documentation. Retrieved September 2026, from <https://docs.python.org/3/library/sqlite3.html>
- Lincoln Loop. (n.d.). qrcode: Python QR code image generator [Computer software]. PyPI. Retrieved September 2026, from <https://pypi.org/project/qrcode/>
- Saifee, A. (n.d.). Flask-Limiter documentation. Retrieved September 2026, from <https://flask-limiter.readthedocs.io/>

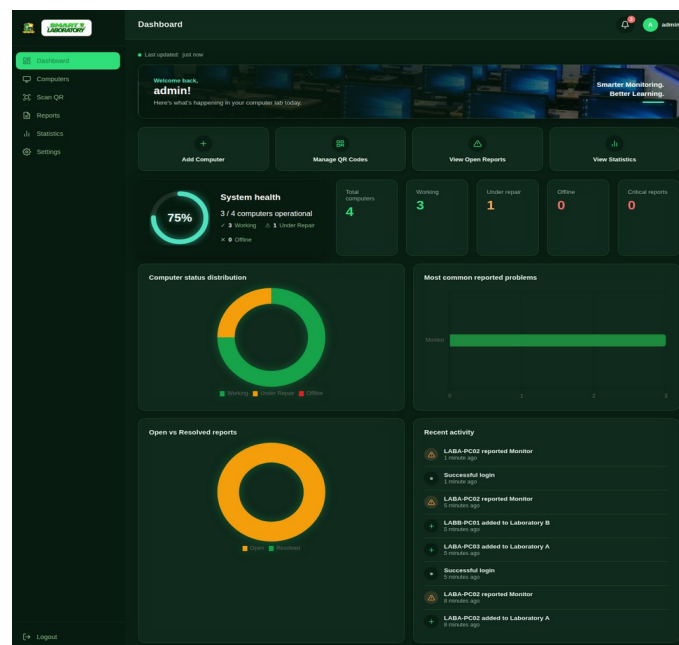
APPENDICES

Appendix A — User Manual

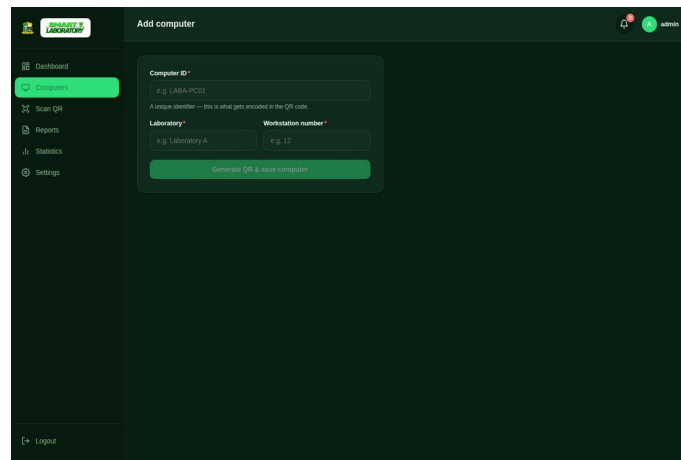
Step 1: Open the application in a web browser. A branded loading screen appears briefly, then log in with the administrator username and password.



Step 2: From the Dashboard, review the live system health bar, the current computer/report counters, and the Critical Reports card, which pulses red whenever a critical-priority report is still open. These update automatically every few seconds — no manual refresh needed.



Step 3: Go to Computers → Add computer. Enter a unique computer ID, its laboratory, and its workstation number, then save. A QR code is generated automatically.

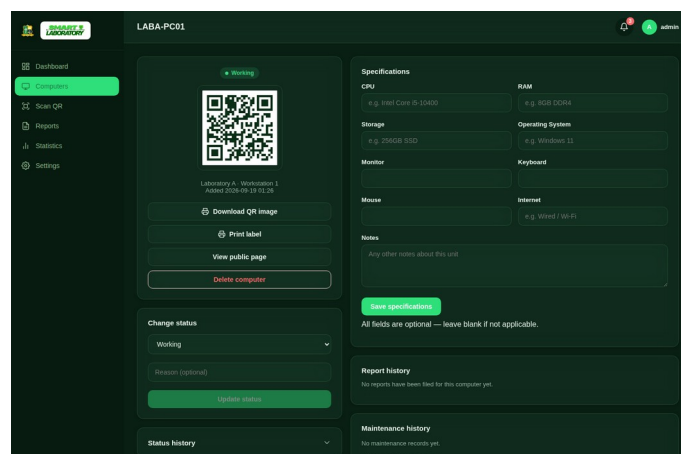


The screenshot shows the 'Add computer' form in the LABVIEW 2.0 interface. The form is titled 'Add computer' and is located in the center of the screen. It contains the following fields and buttons:

- Computer ID***: A text input field with a placeholder 'e.g. LABA-PC01' and a note 'A unique identifier — this is what gets encoded in the QR code.'
- Laboratory***: A text input field with a placeholder 'e.g. Laboratory A'.
- Workstation number***: A text input field with a placeholder 'e.g. 12'.
- Generate QR & save computer**: A green button at the bottom of the form.

The interface also features a sidebar on the left with navigation options: Dashboard, Computers (highlighted), Scan QR, Reports, Statistics, and Settings. The top right corner shows a user profile icon and the name 'admin'.

Step 4: Open that computer's profile to optionally record its specifications (CPU, RAM, storage, and so on), review its status-history timeline, and log maintenance records as work is performed on it.

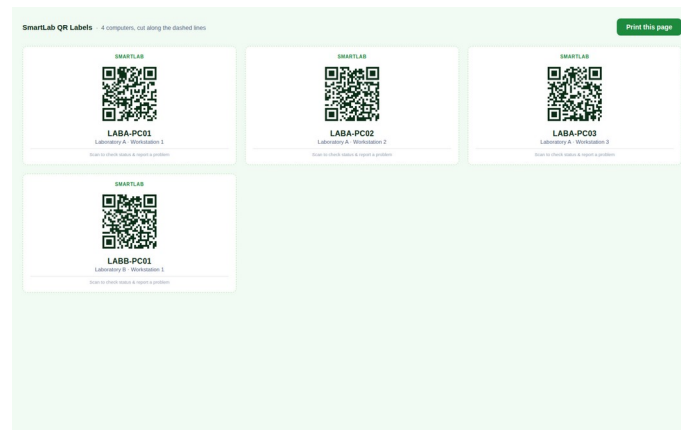


The screenshot shows the computer profile page for 'LABA-PC01' in the LABVIEW 2.0 interface. The page is titled 'LABA-PC01' and is located in the center of the screen. It contains the following sections and fields:

- Working**: A green status indicator at the top.
- QR Code**: A QR code representing the computer's profile.
- Download QR image**: A button to download the QR code.
- Print label**: A button to print the label.
- View public page**: A button to view the public page.
- Delete computer**: A red button to delete the computer.
- Change status**: A dropdown menu with 'Working' selected.
- Reason (optional)**: A text input field.
- Update status**: A green button.
- Status history**: A dropdown menu.
- Specifications**: A section with fields for CPU (e.g. Intel Core i5-10400), RAM (e.g. 8GB DDR4), Storage (e.g. 256GB SSD), Operating System (e.g. Windows 11), Monitor, Keyboard, Mouse, and Internet (e.g. Wired / Wi-Fi).
- Notes**: A text area for notes, with a placeholder 'Any other notes about this unit'.
- Save specifications**: A green button.
- Report history**: A section with a note 'No reports have been filed for this computer yet.'
- Maintenance history**: A section with a note 'No maintenance requests yet.'

The interface also features a sidebar on the left with navigation options: Dashboard, Computers (highlighted), Scan QR, Reports, Statistics, and Settings. The top right corner shows a user profile icon and the name 'admin'.

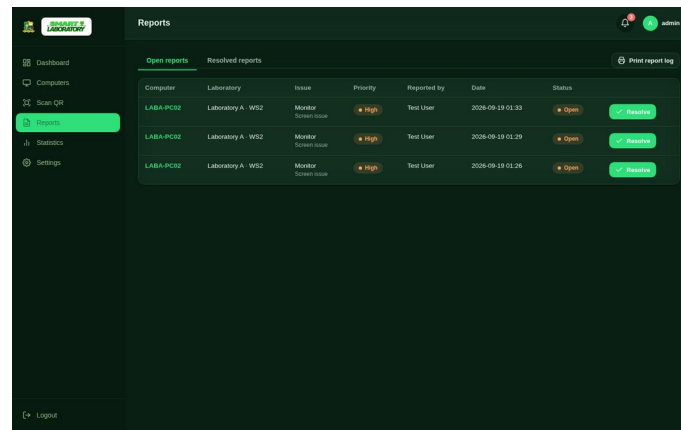
Step 5: Print QR labels for one computer or all of them from Settings → QR Codes → Print QR Codes, and attach them to the physical machines.



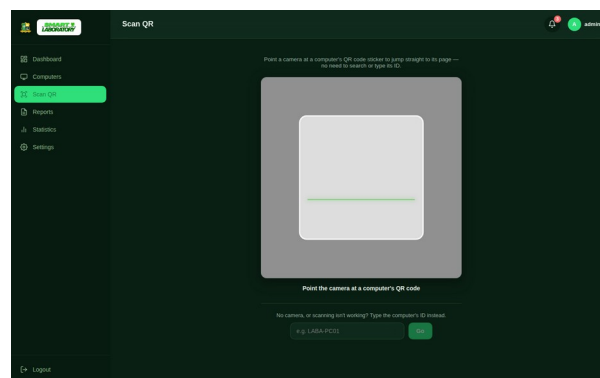
Step 6: Anyone can scan a computer's QR code with a phone camera to open its public status page — no login needed — and submit a categorized problem report from there.

A screenshot of a mobile app interface for a computer's status page. The header shows the "SMARTLAB" logo. The main section is titled "LABA-PC02" with the subtitle "Laboratory A - Workstation 2". Below this, it says "Current status" and shows a green pill with a white dot and the text "Under Repair". At the bottom of this section, it says "Added 2026-09-19 01:26". The next section is titled "Report a problem". It contains three input fields: "Issue type *" with a dropdown menu showing "Select an issue", "Description (optional)" with a text area and placeholder "Briefly describe what's wrong", and "Your name (optional)" with a text field and placeholder "e.g. Juan Dela Cruz". Below these fields is a green "Submit report" button. At the bottom, there is a "Recent reports" section with a "Monitor" button and the timestamp "2026-09-19 01:33", and an "Open" button.

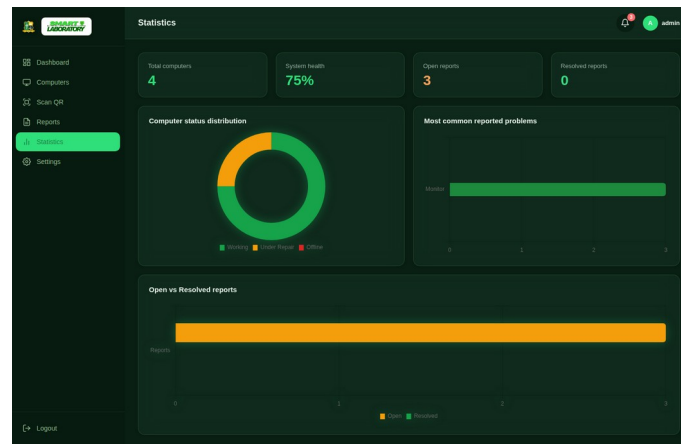
Step 7: New reports appear immediately under Reports and in the notification bell, with a priority badge and their current stage in the Open → Investigating → Under Repair → Resolved workflow. Update the stage as work progresses; reaching Resolved automatically restores the computer's status if no other reports on it remain open.



Step 8: An administrator standing at a computer can also open Scan QR in the sidebar and point their device's camera at that computer's sticker to jump straight to its profile, instead of searching for it manually.



Step 9: Use the Statistics page at any time to review the overall status breakdown and the most frequently reported problems across the whole laboratory, shown as interactive charts.



Appendix B — Team Contribution

Member	Contribution
Marmito, Deric Lloyd A.	Programmer
Villaceran, Ernah Mae G.	Interface Design
Costillas, Bret Hart	Prints
Cuadra, Charles L.	Testing

Defense Milestones

Milestone	Date
Chapters 1–3 Defense	September 22
System Defense	September 24
Chapters 1–5 with System	October 1
Final Defense	October 13